

OPEN SOURCE · ROBOTIK

Unitree G1

KI-Robotik Plattform

ROS 2 · Docker · PyTorch · YOLO · VLM/LLM

Beispielhaftes Architektur- und Umsetzungskonzept eines KI-Robotikprojects aus Sicht eines Systemarchitekten.



Agenda

Übersicht der Präsentation

01 Projektvision & Problemstellung

02 Systemarchitektur

03 Hardware-Stack

04 Software-Stack

05 Datenfluss

06 Live-Demo

07 Docker & Containerisierung

08 GitHub & Projektstruktur

09 Ausblick & Erweiterungen

1. Projektvision

Ziel · Motivation · Anwendungsfall



Problem heute

Engineering Wissen ist auf viele Tools und Dokumente verteilt
Requirements, Architektur, Simulation und Tests sind oft getrennte Prozesse
Änderungen müssen manuell nachgezogen werden
Der Weg von der Idee bis zum lauffähigen Robotersystem ist zeitaufwendig



Vision

Eine KI-gestützte Engineering Platform, die den gesamten Entwicklungsprozess unterstützt: von der Anforderung bis zum laufenden Roboter.



Ziel des Projekts

Entwicklung einer modularen Referenzarchitektur für einen humanoiden Roboter.



Demonstrator

Unitree G1 als Beispielplattform für Entwicklung, Integration und Demonstration

2. Systemarchitektur

7 Cloud / Externe Plattformen
(Nicht Echtzeit)

6 Orchestrierung & Automatisierung
(Workflows)

5 ROS 2
Middleware
(Kommunikation)

4 KI-Frameworks
(Libraries)

3 Containerisierte
Anwendungen
(Docker)

2 Host-Betriebssystem
(Roboter OS)

1 Hardware-Abstraktion
(HAL)

0 Physische
Hardware
(Sensoren & Aktoren)



ENGINEERING & LIFECYCLE TOOLCHAIN (Nicht Echtzeit)



1. REQUIREMENTS MANAGEMENT

- IBM DOORS • Jama Connect • Polarion ALM
- Anforderungen verwalten & versionieren
- Requirement Baselines
- Rückverfolgbarkeit (Traceability)



2. SYSTEM MODELLING (MBSE)

- SysML / UML
- PTC Modeler (formerly Arbortext)
- Cameo Systems Modeler
- IBM Rhapsody
- Enterprise Architect
- Block Definition Diagrams
- Internal Block Diagrams



3. SIMULATION & VALIDATION

- NVIDIA Isaac Sim / Isaac Lab
- Gazebo / Ignition
- MATLAB / Simulink
- Szenarien & digitale Zwillinge
- Algorithmusvalidierung (SIL)



4. CI / CD & DEVOPS

- Git / GitLab
- Docker / Container Registry
- Kubernetes (Orchestrierung)
- Automatisierte Builds & Deployments
- Infrastructure as Code



5. TESTING & VERIFICATION

- Unit Tests
- SIL (Software-in-the-Loop)
- HIL (Hardware-in-the-Loop)
- PIL (Processor-in-the-Loop)
- Integration Tests & Regression Tests



6. DEPLOYMENT & OPERATIONS

- Deployment / Release Management
- OTA (Over-the-Air Updates)
- Monitoring, Logging & Alerting
- Remote Support & Wartung



LEGENDE

- Datenfluss (von unten nach oben)
- ↔ Kommunikation / Austausch (seitlich & vertikal)
- ↔ Externe Verbindung (Internet / Cloud)
- Echtzeit / Sicherheits-Pfad (Hard Real-Time)

KURZ GESAGT

- Hardware liefert Rohdaten → Treiber abstrahieren die Hardware.
- Das Host-Betriebssystem stellt die Basis-Ressourcen bereit.
- Containerisierte Anwendungen führen spezialisierte Aufgaben aus.
- KI-Frameworks ermöglichen die Entwicklung von KI-Modellen.
- ROS 2 verbindet alle Komponenten in Echtzeit.
- nbn/Make automatisiert Prozesse und integriert externe Dienste.
- Cloud-Plattformen bieten Speicher, Training, Management und Interfaces.
- Safety-Ebene garantiert sichere Reaktion unabhängig vom restlichen Stack.

8-Schichten-Modell
von Hardware bis Cloud

Simplified public version

2.1 Architekturschichten

Warum Docker, ROS 2 und Kubernetes?

0-1

Hardware & HAL

Sensoren (Kamera, LiDAR, IMU) → Treiber abstrahieren die Hardware

2

Host-OS

Ubuntu 22.04 LTS auf Jetson/PC – Basis für Docker & NVIDIA

3

Docker Container

Isolation, Reproduzierbarkeit, unabhängige Updates pro Modul

4

KI-Frameworks

PyTorch, TensorFlow, ONNX, OpenCV – innerhalb der Container

5

ROS 2 Middleware

DDS-basierte Echtzeit-Kommunikation via Topics, Services, Actions

6

Orchestrierung

n8n / Make – automatisierte Workflows, Trigger, Benachrichtigungen

7

Cloud

Azure/AWS – Training, Langzeitspeicher, Fleet Management, Remote-Update

3. Hardware-Stack

Sensoren, Aktoren & Recheneinheiten



Kamera

- ▶ RGB-D / Depth Kamera
- ▶ 30–60 fps
- ▶ Objekt- & Tiefenerkennung
- ▶ ROS 2 Image Topic



IMU

- ▶ 9-DOF Inertialsensor
- ▶ Beschleunigung & Gyroskop
- ▶ Orientierungsschätzung
- ▶ Echtzeit via CAN/Serial



LiDAR (opt.)

- ▶ 360° Punktwolke
- ▶ SLAM & Kartierung
- ▶ Hinderniserkennung
- ▶ pointcloud2 Topic



AI Computer

- ▶ NVIDIA Jetson / RTX GPU
- ▶ CUDA für PyTorch/YOLO
- ▶ Docker Host
- ▶ NTP Zeitsynchronisation

4. Software-Stack

Betriebssystem

Ubuntu 22.04 LTSm
OS

Robotik

ROS 2 (Humble / Jazzy)
Robotik Framework

Programmierung

Python 3.10+
Programmiersprache

KI / ML

PyTorch *KI Framework*
YOLO v8/v11 *KI Modell*

Vision

OpenCV 4.x
Computer Vision Library

Container

Docker + Compose
Container Platform

Laufzeit

ONNX Runtime
Inference Runtime

GPU Compute

CUDA / CuPy
GPU Compute Platform

5. Datenfluss

Von der Kamera zum Roboter-Befehl



Kamera → YOLO:

OpenCV liest den Kamera-Stream. YOLO v8 führt Echtzeit-Inferenz durch und gibt Bounding Boxes mit Confidence-Werten zurück.

YOLO → ROS 2:

Detektionen werden als strukturierte Nachrichten auf dem /detection/objects Topic publiziert (sensor_msgs / custom msg).

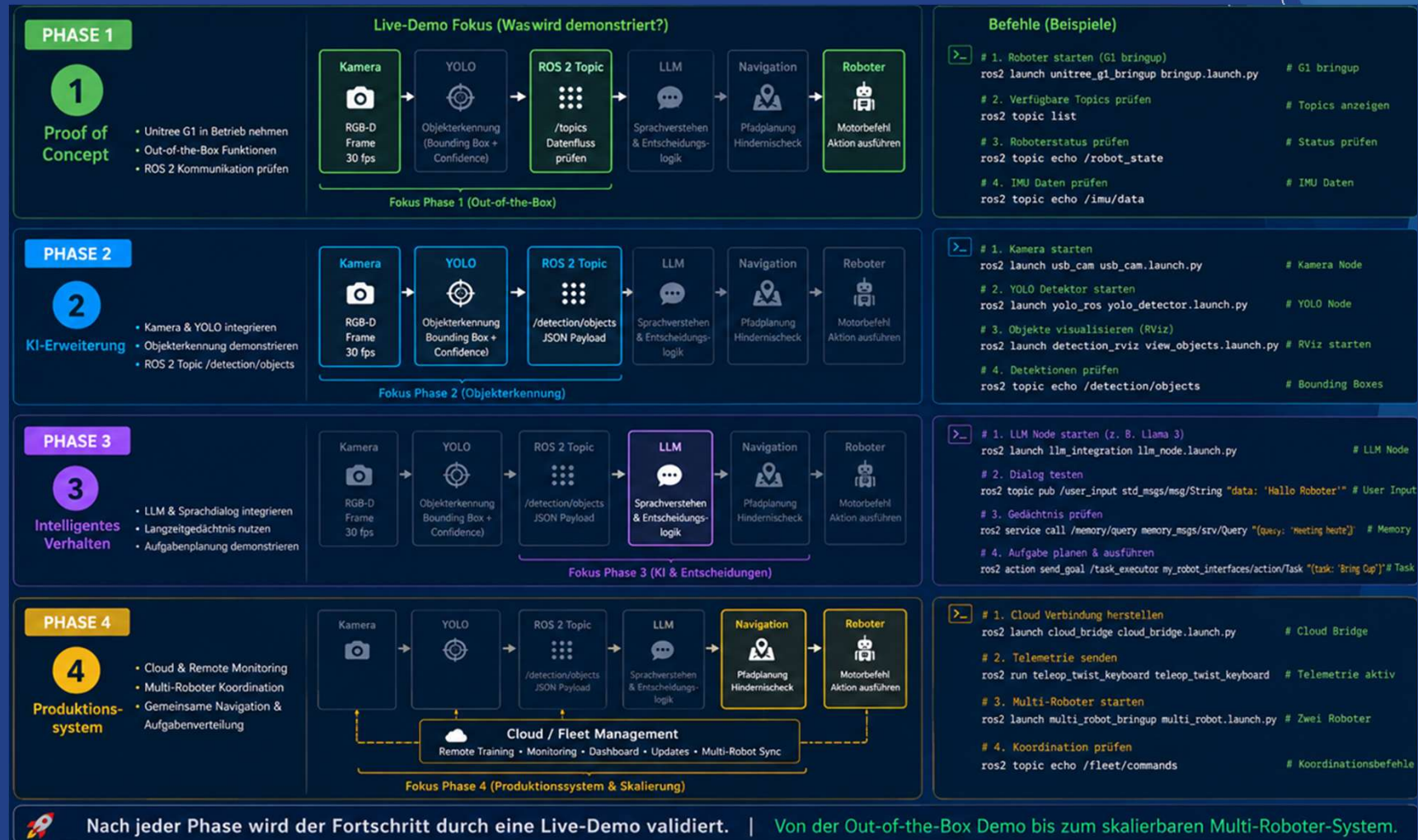
ROS 2 → LLM:

Der LLM-Container abonniert das Topic, verarbeitet Kontext und generiert Handlungsempfehlungen (Planen, Dialogsystem).

LLM → Roboter:

Navigations-Container empfängt Ziel, plant Pfad via SLAM und sendet Gelenkbefehle an die Motorsteuerung.

6. Mögliche Live-Demos je Entwicklungsphase



ROS 2 Terminal

Nach jeder Phase wird der Fortschritt durch eine Live-Demo validiert. | Von der Out-of-the-Box Demo bis zum skalierbaren Multi-Roboter-System.

7. Docker & Containerisierung

Warum Container? · Struktur · Vorteile

Isolation

Jeder Dienst (Sensor, KI, Navigation) läuft in eigenem Container. Kein Abhängigkeitskonflikt.

Reproduzierbarkeit

Einmal bauen, überall ausführen – auf Jetson, Laptop oder Cloud.

Updates

Einzelne Container können unabhängig aktualisiert oder ersetzt werden.

1

sensor-container

Kamera, LiDAR, IMU Nodes

2

vision-container

YOLO, OpenCV, Tracking

3

nav-container

SLAM, Pfadplanung

4

llm-container

Sprachverstehen, Dialog

5

db-container

Langzeitgedächtnis, Vektoren

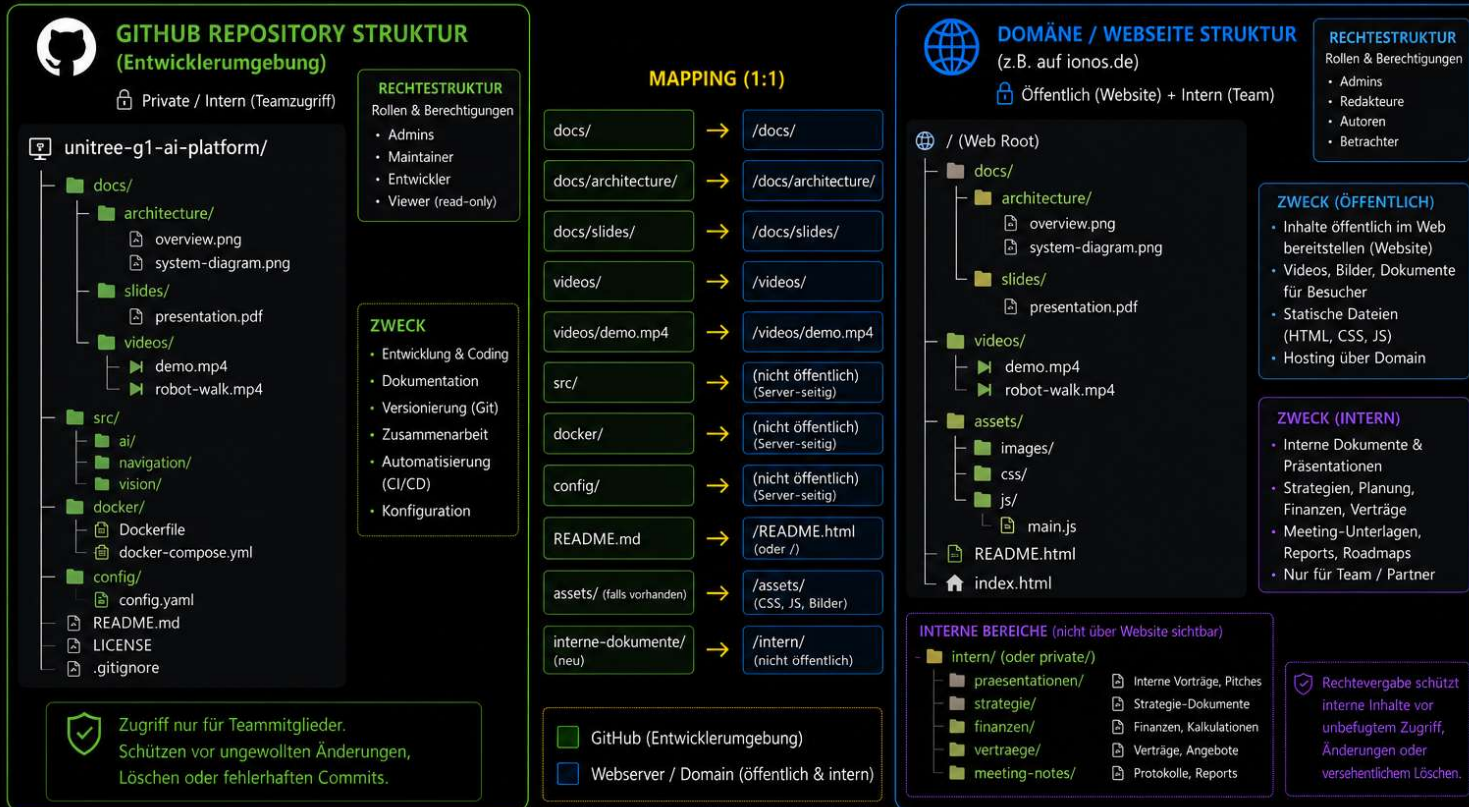
6

monitor-container

RViz, Prometheus, Grafana

8. GitHub/ Domäne & Projektstruktur

VERGLEICH: GITHUB STRUKTUR vs. DOMÄNEN-/WEBSEITE STRUKTUR



EMPFEHLUNG

Git für die Entwicklung, Dokumentation und Versionskontrolle.

Domain für die Veröffentlichung der Inhalte (Webseite, Videos, Downloads).

9. Ausblick & Erweiterungen

Nächste Schritte auf der Roadmap

Phase 1



Grundsystem

ROS2 Workspace, Docker
Container, HW & SW Integration,
Basis Datenfluss Implementierung



Demonstrator

Roboter im Betrieb nehmen (Out-of-Box)
GitHub Repository
Systemarchitektur Dokumentation
Präsentation & Dokumentation

Phase 2



Eigenes YOLO-Training

Custom Dataset für spezifische
Objekte & Personen annotieren
und trainieren



LLM Integration

Lokales LLM (Llama 3 / Mistral) für
Dialogsystem und Aufgabenplanung

Phase 3



Langzeitgedächtnis

Vektor-Datenbank
(Qdrant/Weaviate) für persistente
Nutzerprofile & Erfahrungen



Kubernetes

K3s für automatisches Scaling,
Failover und Rolling Updates auf
dem Roboter

Phase 4



Cloud Integration

Azure/AWS: Remote-Training,
Fleet Management, Monitoring
Dashboard



Multi-Roboter

Koordiniertes Schwarm-Verhalten
mit geteiltem Kartenmaterial und
Aufgabenverteilung



Von der Proof-of-Concept Demo zum vollautonomen, skalierbaren Robotik-System.